

Success Library - Design Decisions

This document records the architectural and design decisions made during the development of the Success Library. Each decision includes context, options considered, and rationale.

Table of Contents

- 1. [Code File Format](#1-code-file-format)
- 2. [HTML Generation Strategy](#2-html-generation-strategy)
- 3. [Index Display Format](#3-index-display-format)
- 4. [Version Control Approach](#4-version-control-approach)
- 5. [Dual Code Implementations (SQL + PySpark)](#5-dual-code-implementations-sql--pyspark)
- 6. [Metadata Management](#6-metadata-management)
- 7. [Directory Structure](#7-directory-structure)

1. Code File Format

Decision: Single markdown file (.md) per metric containing both SQL and PySpark code.

Date: 2026-01-16

Context

Each success metric needs calculation logic. Initially, we created separate .sql files. However, we also need PySpark implementations for Hive access.

Options Considered

Option	Description	Pros	Cons
A. Separate files	VVD_ACQ_001.sql + VVD_ACQ_001.py	Syntax highlighting, can open in SQL tools	2x files, harder to keep in sync
B. Single text file	VVD_ACQ_001.txt with delimiters	One file per metric	No syntax highlighting
C. Single markdown	VVD_ACQ_001.md with code blocks	One file, syntax highlighting on GitHub, hu	Requires markdown convention

Decision

Option C: Single markdown file per metric.

Rationale

- One file per metric reduces maintenance burden
- Markdown code blocks provide syntax highlighting when viewed on GitHub
- SQL and PySpark logic for the same metric stays together
- Scales better as library grows (hundreds of metrics = hundreds of files, not thousands)

Format

```
# METRIC_ID - Metric Name
<!-- Owner: Name | Version: v1.0 -->
```

```
## SQL (Data Warehouse)

SELECT ...

## PySpark (Hive)

df = spark.sql(...)
```

2. HTML Generation Strategy

Decision: Build script generates static HTML from JSON + code files.

Date: 2026-01-16

Context

The HTML index needs to display metric metadata and code. Initially, this was hardcoded directly in the HTML file, duplicating data from the JSON and code files.

Options Considered

Option	Description	Pros	Cons
A. Hardcoded HTML	All data embedded in HTML	Simple, works offline	Duplication, hard to maintain at scale
B. Runtime fetch	JavaScript fetches JSON + code files	Single source of truth	Requires web server (no file://)
C. Build script	Python script reads sources, generates HTML	Single source of truth, works offline	Requires running build after changes
D. Link to files	HTML links to code files (no preview)	Simplest	No inline code viewing

Decision

Option C: Build script (build.py) generates index.html.

Rationale

- Single source of truth: edit JSON and code files, not HTML
- Works offline (static HTML output)
- No duplication or sync issues
- Scales to hundreds of metrics without HTML file becoming unmanageable
- Build step is simple: run python build.py

Workflow

```
1. Edit metadata/success_library_index.json (or code files)
2. Run: python build.py
3. index.html is regenerated
```

3. Index Display Format

Decision: Table format with search and filters.

Date: 2026-01-16

Context

Initial implementation used a card-based layout where each metric was displayed as a card with all metadata fields. This was verbose and repetitive.

Options Considered

Option	Description	Pros	Cons
A. Card layout	Each metric as a visual card	Visually distinct	Verbose, doesn't scale, lots of scrolling
B. Table layout	Standard HTML table with columns	Compact, scannable, professional	Less visual distinction between items

Decision

Option B: Table layout.

Rationale

- Scales better: 300+ metrics need compact display
- Professional appearance
- Easier to scan and compare metrics
- Standard pattern users are familiar with
- Can still expand rows or open modal for full details + code

Table Columns

Based on metadata attributes defined in session notes:

- Metric ID
- Metric Name
- Product
- Line of Business
- Metric Type
- Pillar
- Business Definition
- Source Tables
- Staged Source
- Grain
- Measurement Window
- Owner
- Version
- Campaigns Using
- Code (link/expand)

4. Version Control Approach

Decision: Git-based versioning with no review gate (Option B).

Date: 2026-01-16

Context

Need to track changes to code files and metadata. Also need to decide on governance level.

Options Considered

Option	Description	Pros	Cons
A. Pull Request workflow	Changes require review before merge	Governance, catches errors	Slower, requires reviewers
B. Direct commits	Anyone with access commits directly	Fast, simple	No approval step
C. Manual version tracking	Version numbers in files only	No Git dependency	Error-prone, no real history

Decision

Option B: Direct commits (no review gate).

Rationale

- Simplicity: don't want to slow down metric updates
- Git provides full history anyway (who, when, what)
- Can always rollback if something breaks
- Team is small enough to trust direct commits
- Can add review gate later if governance needs increase

Version Tracking

- Git: Handles detailed versioning automatically (every commit tracked)
- Manual label: Version: v1.0 in file header for major revisions only
- No need to maintain version numbers manually for every small change

5. Dual Code Implementations (SQL + PySpark)

Decision: Maintain two separate code implementations per metric, not wrappers.

Date: 2026-01-16

Context

The organization has two data environments:

1. Data Warehouse (Relational DB): Accessed via pure SQL (Teradata now, Snowflake future)
2. Hive (Data Lake): Accessed via PySpark through YARN/Spark

Initially considered having PySpark wrap the SQL code. This doesn't work because the environments have different table paths.

Why Not Wrappers

```
# This WON'T work:
spark.sql("SELECT ... FROM DGNV01.TACTIC_EVNT_IP_AR_HIST")
# ? Data Warehouse path doesn't exist in Hive environment
```

The same logical table exists in both environments but with different paths:

- Data Warehouse: DGNV01.TACTIC_EVNT_IP_AR_HIST
- Hive: hive_analytics.tactic_evnt_hist (or HDFS path)

Decision

Two dedicated implementations per metric.

Each code file contains:

1. SQL implementation (for Data Warehouse queries)
2. PySpark implementation (for Hive queries)

These are independent implementations of the same business logic, not one derived from the other.

6. Metadata Management

Decision: JSON file as source of truth, with future Excel intake process.

Date: 2026-01-16

Current Approach

- metadata/success_library_index.json contains all metric metadata
- Manually edited (for now)

Future Approach (Not Yet Implemented)

- Excel template for analysts to fill out
- Script converts Excel to JSON
- Prevents manual JSON editing errors

Rationale for Future Excel Intake

- Analysts are comfortable with Excel
- JSON syntax is error-prone for non-developers
- Excel can have validation, dropdowns, formatting
- Script ensures consistent JSON output

Metadata Attributes

Attribute	Description
metric_id	Unique identifier (e.g., VVD_ACQ_001)
metric_name	Human-readable name
product	Product category (VVD, Credit Card, etc.)
line_of_business	LOB grouping
metric_type	Acquisition / Activation / Usage / Provisioning
pillar	Conversion / Engagement / Retention / etc.
business_definition	Plain English description
code_path	Path to code file (e.g., code/VVD/VVD_ACQ_001.md)
source_tables	Raw data sources
staged_source	Curated/pre-calculated source (if exists)
grain	Client / Account / Transaction
measurement_window_days	Days post-treatment
owner	Responsible person
version	Manual version label
campaigns_using	List of campaigns using this metric

7. Directory Structure

Decision: Organized by function with clear separation of concerns.

Date: 2026-01-16

Structure

```
NBA Souccess Library/
|-- README.md           # Project overview, quick start
|-- index.html          # GENERATED - do not edit directly
|-- build.py            # Generates index.html
|
|-- metadata/
|   |-- success_library_index.json  # Metric metadata (source of truth)
|
|-- code/
|   |-- old/              # Legacy/reference code
|   |-- {PRODUCT}/        # Organized by product
|       |-- {METRIC_ID}.md    # Combined SQL + PySpark
|
|-- docs/
|   |-- DESIGN_DECISIONS.md    # This file
|
|-- support/
|   |-- success_library_project_context.md  # Full project specification
|
|-- agent_sessions/
|   |-- YYYY-MM-DD_session.md    # Working session notes
```

Rationale

- code/ organized by product for scalability (VVD/, CC/, MORTGAGE/, etc.)
- metadata/ separate from code for clarity
- docs/ for design documentation
- support/ for project context and reference material
- agent_sessions/ preserves working notes and decision history

8. Intake Workflow (Inbox/Processed Pattern)

Decision: Use inbox/processed folder pattern with single intake form supporting both new and update actions.

Date: 2026-01-16

Context

Multiple analysts need to submit new metrics or updates. Need a safe, auditable process that prevents data loss.

Folder Structure

```
intake/
|-- pending/           ? Analysts drop files here
|-- processed/         ? Script moves files here after processing
|-- template/
|   |-- intake_template.xlsx  ? Blank template
```

Intake Form Design

Single Excel form with action column:

- new - Adding a metric that doesn't exist
- update - Modifying an existing metric (empty fields = keep existing)

Script Behavior (excel_to_json.py)

1. Find all .xlsx files in intake/pending/
2. Read and validate entries
3. Create backup: metadata/backups/success_library_index_backup_TIMESTAMP.json
4. Show summary of changes (what will be added/updated)
5. Ask for confirmation: "Apply these changes? (yes/no)"
6. If yes: update JSON, move files to intake/processed/

Safety Features

- Backup created before any change
 - Validation prevents invalid entries
 - Human confirmation required
 - Processed files preserved (audit trail)
 - Git provides additional version control
-

9. Separate Build Scripts

Decision: Keep build.py and excel_to_json.py as separate scripts.

Date: 2026-01-16

Context

We have two types of updates:

1. Code-only updates (change SQL/PySpark logic in .md files)
2. Metadata updates (add new metrics, change owners, definitions, etc.)

Options Considered

Option	Description	Pros	Cons
A. Combined script	One script handles Excel intake + HTML g	One command	Must handle empty Excel, complex logic
B. Separate scripts	excel_to_json.py for metadata, build.py for	Simple, clear purpose	Two commands for full update

Decision

Option B: Separate scripts.

Rationale

- Code-only updates are common - Analysts update SQL logic without changing metadata. Only need build.py.
- Single responsibility - Each script does one job well.
- No empty form handling - Don't need logic to detect/skip empty Excel files.

- Easier debugging - If something breaks, clear which script is responsible.

Workflow Summary

Scenario: Updated code logic only
-> python3 build.py

Scenario: Added new metric or updated metadata
-> python3 excel_to_json.py
-> python3 build.py

Change Log

Date	Decision	Section
2026-01-15	Initial architecture (cards, SQL only)	-
2026-01-16	Table format, build script, combined .md files	1, 2, 3
2026-01-16	Git versioning, no review gate	4
2026-01-16	Dual SQL + PySpark implementations	5
2026-01-16	Intake workflow (inbox/processed pattern)	8
2026-01-16	Separate build scripts	9

Document created: 2026-01-16 Last updated: 2026-01-16